0ICPC455

AY 2024-25

Machine Learning Lab

Quality Laboratory Manual

Experiment No. 5

Study and Implementation of Decision Tree Algorithm

Course Instructor -

DR. TAHSEEN A. MULLA

ASSOCIATE PROFESSOR

Experiment No. 5

Title of Experiment: To study and implement Decision Tree Algorithm

Aim of Experiment: To implement and understand the working of Decision Tree algorithm, a supervised learning technique used for both classification and regression taska in Machine Learning

System Requirements – Win 8 and above OS, 4GB RAM, 2.33 GHz Processor

Software/s Needed for Experiment – Jupyter Notebook/ Anaconda Navigator/ Google Colaboratory/ Spyder, Python 3.x [With libraries such as Numpy, Pandas, Matplotlib and Scikit-Learn]

Experiment Outcomes –

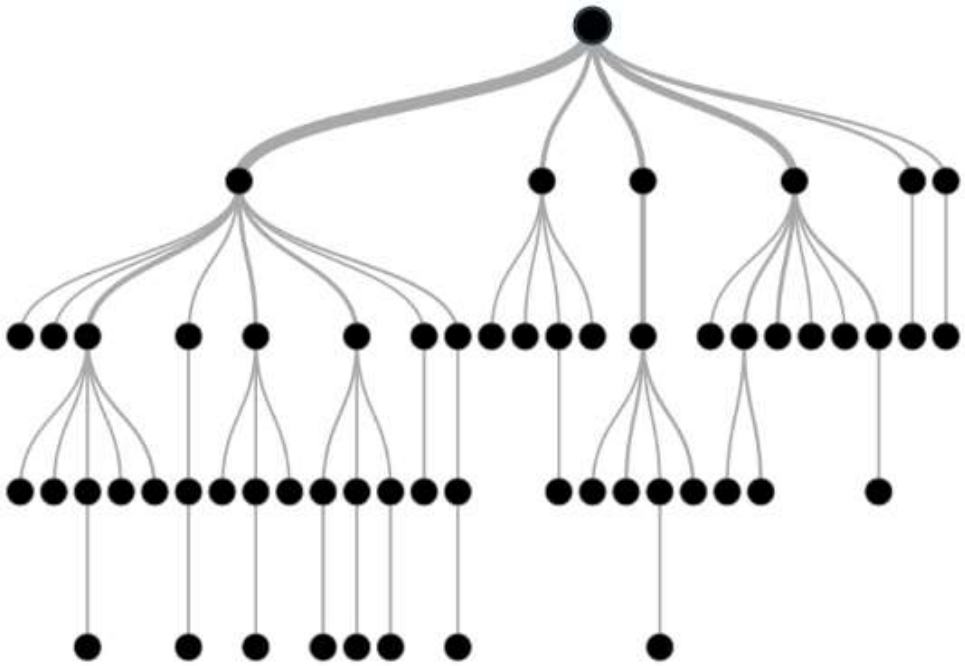
1. Understand the concept and principles of Decision Tree algorithm
2. To understand and implement the structure and components of a decision tree, including root nodes, internal nodes, branches and leaf nodes
3. Able to evaluate the performance of decision tree using appropriate metrics
4. To explain the decision-making process of the model based on the visualization

Theory –

Decision Tree is a Supervised learning technique that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome.

In a Decision tree, there are two nodes, which are the Decision Node and Leaf Node. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.

The decisions or the test are performed on the basis of features of the given dataset. It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions. It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.



How Decision Tree is formed?

The process of forming a decision tree involves recursively partitioning the data based on the values of different attributes. The algorithm selects the best attribute to split the data at each internal node, based on certain criteria such as information gain or Gini impurity. This splitting process continues until a stopping criterion is met, such as reaching a maximum depth or having a minimum number of instances in a leaf node.

Why Decision Tree?

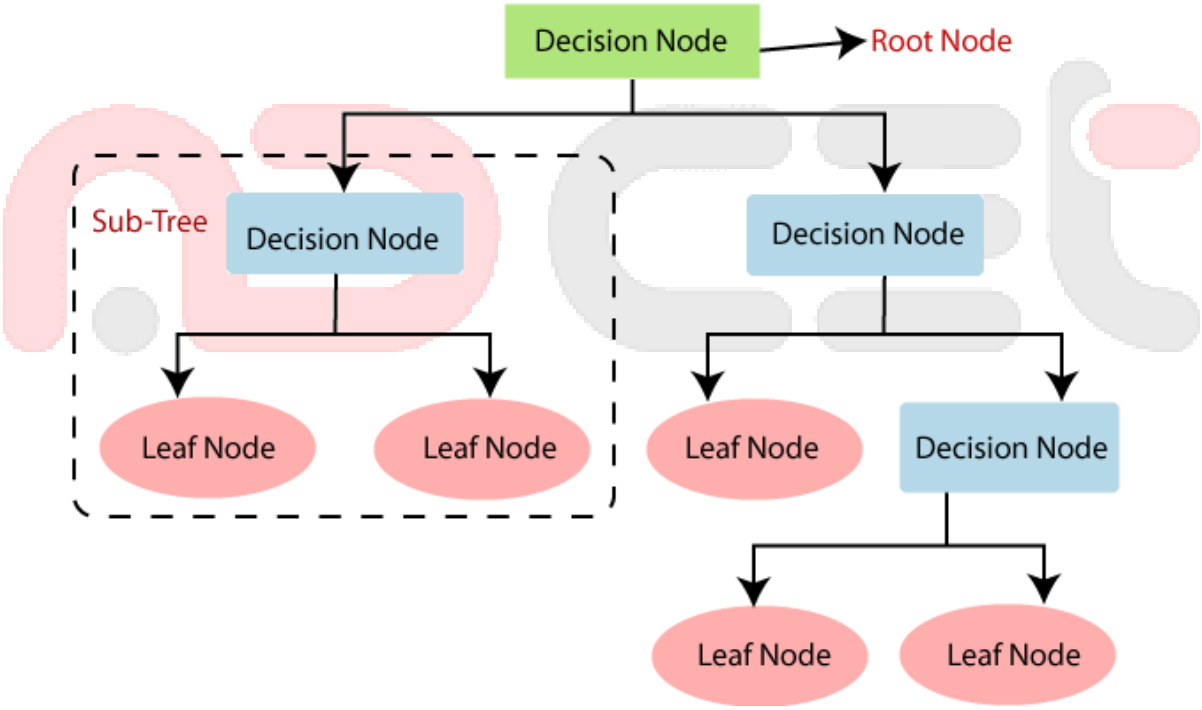
Decision trees are widely used in machine learning for a number of reasons:

- Decision trees are so versatile in simulating intricate decision-making processes, because of their interpretability and versatility.
- Their portrayal of complex choice scenarios that take into account a variety of causes and outcomes is made possible by their hierarchical structure.
- They provide comprehensible insights into the decision logic, decision trees are especially helpful for tasks involving categorization and regression.
- They are proficient with both numerical and categorical data, and they can easily adapt to a variety of datasets thanks to their autonomous feature selection capability.

- Decision trees also provide simple visualization, which helps to comprehend and elucidate the underlying decision processes in a model.

Decision Trees are tree-structured models that make decisions based on the input features. Each internal node represents a "decision" on an attribute, each branch represents the outcome of the decision, and each leaf node represents a class label (for classification) or a continuous value (for regression).

- **Root Node:** Represents the entire dataset, which is then split into subsets based on an attribute.
- **Internal Nodes:** Represent the attributes on which the data is split.
- **Leaf Nodes:** Represent the outcome or class labels



The splitting of nodes is based on criteria like Gini Index or Information Gain (using Entropy). The goal is to create branches that lead to the most homogenous subsets, reducing the impurity at each step.

Entropy and Information Gain are calculated as:

Entropy (E):

$$E(S) = -\sum_{i=1}^{c} p_i \log_2 (p_i)$$

Where;
p_i is the proportion of the dataset belonging to class i

Information Gain (IG):

$$IG(S, A) = E(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \times E(S_v)$$

Where;
S_v is the subset of S where attribute A has value v

Procedure to implement Logistic Regression:

1. **Data Collection:** Obtain a dataset suitable for classification or regression. A dataset such as Iris, which classifies iris flowers into three species – Setosa, Versicolour and Virginica

2. **Data Preprocessing:**
   a. Load the dataset into a Pandas DataFrame.
   b. Handle missing values, outliers, and encode categorical variables if necessary.

3. **Exploratory Data Analysis (EDA):** Visualize relationships between the features and the target class using scatter plots, pair plots and correlation matrices

4. **Splitting the Data:** Split the dataset into training and test sets to evaluate the model's performance.

5. **Implementing Decision Tree Algorithm:**
   a. Use the Scikit-learn library to implement decision tree model.
   b. Train the model using the training data.

6. **Model Prediction:**
   a. Use the trained model to make predictions on the test data.
   b. Generate a confusion matrix and classification report to assess accuracy

7. **Model Evaluation:**
   a. Evaluate the model using metrics such as accuracy, precision, recall, F1-score for each class and the overall weighted average
   b. Visualize the decision tree structure

Sample Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix,
classification_report
from sklearn.datasets import load_iris

# Step 1: Data Collection (Iris dataset)
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.DataFrame(iris.target, columns=['species'])

# Step 2: Data Preprocessing
# The Iris dataset is already clean with no missing values or
outliers.

# Step 3: Exploratory Data Analysis (EDA)
sns.pairplot(pd.concat([X,     y],     axis=1),     hue='species',
palette='Set1')
plt.show()

# Step 4: Splitting the Data
X_train,  X_test,  y_train,  y_test  =  train_test_split(X,  y,
test_size=0.2, random_state=42)

# Step 5: Implementing the Decision Tree Algorithm
dtree        =        DecisionTreeClassifier(criterion='entropy',
random_state=42)
dtree.fit(X_train, y_train)

# Step 6: Model Prediction
y_pred = dtree.predict(X_test)

# Comparing Actual vs Predicted
comparison_df  =  pd.DataFrame({'Actual':  y_test.values.ravel(),
'Predicted': y_pred})
print(comparison_df)

# Step 7: Model Evaluation
# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print(f"Confusion Matrix:\n{conf_matrix}")
```

```
# Classification Report
class_report = classification_report(y_test, y_pred,
target_names=iris.target_names)
print(f"Classification Report:\n{class_report}")

# Visualizing the Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(dtree, feature_names=iris.feature_names,
class_names=iris.target_names, filled=True)
plt.title("Decision Tree")
plt.show()
```

Observations -

- Record the predicted classes for the test data
- Observe the performance of the model using the confusion matrix and classification report

Conclusion –

Hence, the model summarizes the findings from the experiment, such as effectiveness of the Decision Tree model in classifying iris species

References –

a. Textbook –

i. Machine Learning with Python – An approach to Applied ML – Abhishek Vijayvargiya, BPB Publications, 1st Edition 2018

ii. Machine Learning, Tom Mitchell, McGraw Hill Education, 1st Edition 1997

b. Online references –

i. https://scikit-learn.org/stable/modules/tree.html

ii. https://www.hackerearth.com/practice/machine-learning/machine-learning-algorithms/ml-decision-tree/tutorial/

iii. https://www.coursera.org/in/articles/decision-tree-machine-learning

Expected Oral Questions –

1. What is a decision tree and how does it work?
2. What are the main components of a decision tree?
3. How is a decision tree different from other machine learning algorithms?
4. What types of problems are decision trees typically used for?
5. What are the key advantages and disadvantages of using a decision tree?
6. How is a decision tree constructed? What are the steps involved in it?
7. Explain the concept of information gain and how it is used in decision tree?
8. What are Gini impurity and Entropy and how are they used in decision tree algorithm?

9. How do you interpret the structure of a decision tree?
10. How do decision trees handle categorical v/s continuous variables?

FAQ's in Interview –

1. List down some popular algorithms used for deriving Decision Trees and their attribute selection measures
2. Briefly explain the properties of Gini impurity
3. Explain the difference between the CART and ID3 algorithms
4. List down the different types of nodes in Decision Trees
5. Are Decision Trees affected by the outliers?